

RISC-V VeeR EL2 Based Hardware Packet Buffering and Data Integrity SoC

Name: V Sai Charan Goud

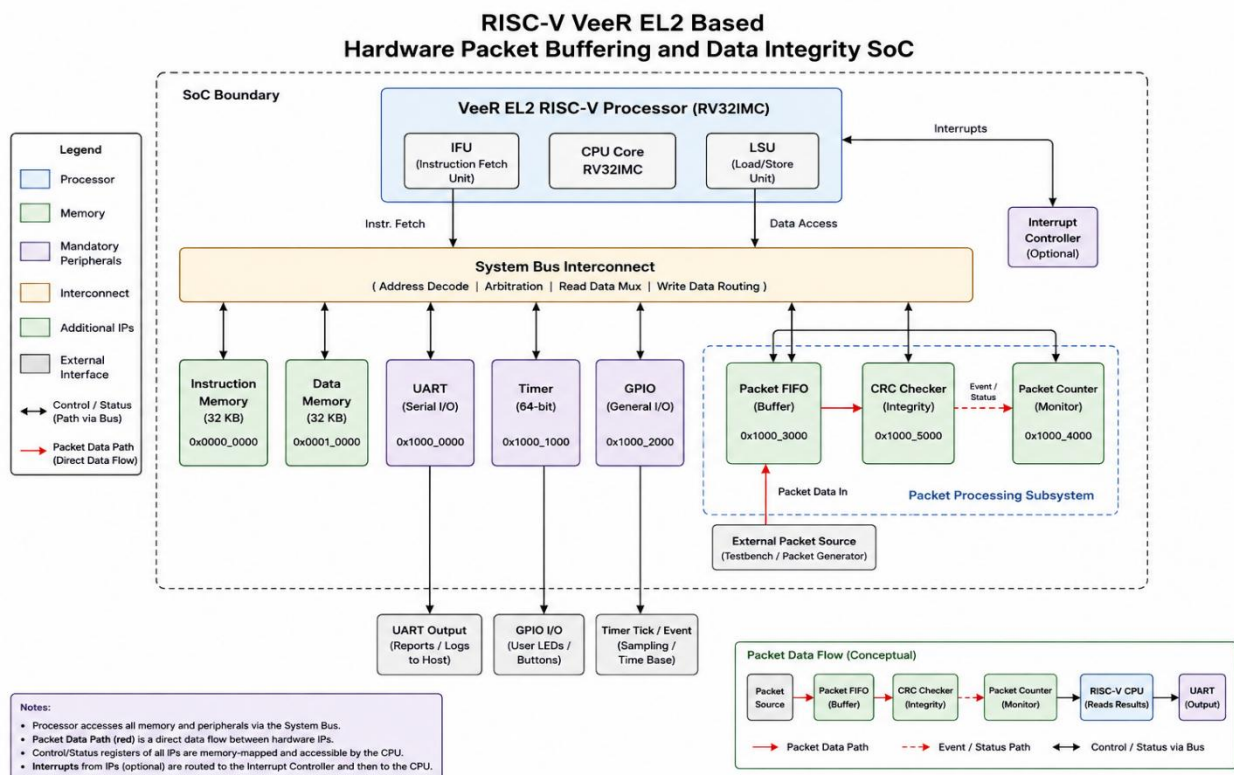
Roll Number: 1602-23-735-316

1. System Architecture

The proposed SoC is organized around the VeeR EL2 RISC-V processor as the central processing unit, connected to instruction memory, data memory, mandatory peripherals, and additional IP blocks through a shared system bus. The mandatory peripherals include UART, Timer, and GPIO, while the three additional IPs are Packet FIFO, Packet Counter, and CRC Checker. The Packet FIFO buffers incoming packet data, the Packet Counter maintains packet statistics, and the CRC Checker verifies packet integrity. The processor accesses the memories and IPs through memory-mapped registers and controls the overall packet-processing operation. Packet data is supplied by a testbench packet source, processed by the hardware IPs, and the resulting status and verification information is read by the RISC-V processor and reported through UART.

2. Block Diagram

The block diagram shows the VeeR EL2 RISC-V processor connected to memories, mandatory peripherals, and the three additional IPs through a system bus.



The packet-processing path is Packet Source → Packet FIFO → CRC Checker, while the Packet Counter monitors activity and the processor reports the results through UART.

Block Diagram Description

2.1 VeeR EL2 RISC-V Processor

The VeeR EL2 processor is the central processing unit of the SoC. It executes the application, controls the peripherals, and reads packet and CRC status.

2.2 System Bus Interconnect

The system bus connects the processor, memories, mandatory peripherals, and additional IPs. It performs address decoding and routes read/write transactions.

2.3 Instruction Memory

Instruction Memory stores the program executed by the VeeR EL2 processor.

2.4 Data Memory

Data Memory stores application data, variables, and temporary processing results.

2.5 UART

UART provides serial output for displaying packet status, CRC results, and error information.

2.6 Timer

The Timer provides timing functionality and periodic timing events for the application.

2.7 GPIO

GPIO provides general-purpose digital input/output and can be used for basic status indication.

2.8 Packet FIFO

The Packet FIFO buffers incoming packet data before processing. It provides empty, full, and overflow status information.

2.9 Packet Counter

The Packet Counter maintains packet statistics such as received packets, processed packets, CRC errors, and FIFO overflows.

2.10 CRC Checker

The CRC Checker verifies packet integrity by calculating and comparing the CRC value to detect corrupted packets.

2.11 External Packet Source

The External Packet Source is a testbench-generated packet stream used to verify the packet-processing system.

2.12 Packet Processing Flow

The packet flow is **Packet Source → Packet FIFO → CRC Checker**, while the Packet Counter monitors packet activity. The processor reads the results through the system bus and reports them through UART.

3. Memory Mapping

The SoC uses a memory-mapped architecture in which the processor accesses memories and peripherals using dedicated address ranges. The system bus decodes the processor address and selects the corresponding memory or peripheral. Each block is assigned a separate address region to avoid address conflicts and simplify peripheral integration.

Sl. No.	Block	Base Address (Hex)	End Address (Hex)	Size	Description
1	Instruction Memory	0x0000_0000	0x0000_7FFF	32 KB	Stores program instructions for the processor.
2	Data Memory	0x0001_0000	0x0001_7FFF	32 KB	Stores program data, variables and intermediate results.
3	UART	0x1000_0000	0x1000_0FFF	4 KB	Serial communication interface for reporting results and logs.
4	Timer	0x1000_1000	0x1000_1FFF	4 KB	Provides timing functions and generates periodic events.
5	GPIO	0x1000_2000	0x1000_2FFF	4 KB	General purpose digital input/output interface.
6	Packet FIFO	0x1000_3000	0x1000_3FFF	4 KB	Buffers incoming packet data and provides status flags.
7	Packet Counter	0x1000_4000	0x1000_4FFF	4 KB	Maintains packet statistics such as counts and error events.
8	CRC Checker	0x1000_5000	0x1000_5FFF	4 KB	Computes and verifies CRC for packet integrity checking.

The Packet FIFO, Packet Counter, and CRC Checker are accessed through their respective memory-mapped register interfaces. The final base addresses and memory sizes will be confirmed after inspecting the supplied VeeR EL2 RTL configuration and resolving any conflicts with processor-local memory regions. The reference architecture similarly uses aligned peripheral address windows and verifies the supplied VeeR configuration before finalizing the map.

4. Interface Specifications

4.1 RISC-V / System Bus Interface

- The VeeR EL2 RISC-V processor acts as the main master for instruction and data transactions.
- The system interconnect decodes the address and routes each transaction to Instruction Memory, Data Memory, UART, Timer, GPIO, Packet FIFO, Packet Counter, or CRC Checker.

- The processor accesses peripheral and IP registers through memory-mapped read and write operations.
- The exact bus protocol, signal names, and handshake mechanism will be finalized according to the supplied VeeR EL2 RTL configuration.

4.2 Packet FIFO Interface

Interface	Signals / Function
System side	Memory-mapped register interface for FIFO control and status
Packet input	DATA_IN, VALID_IN, READY_IN
Packet output	DATA_OUT, VALID_OUT, READY_OUT
Control	ENABLE, RESET, READ, WRITE
Status	EMPTY, FULL, OVERFLOW
Data	Packet data stream
Optional	Packet-available / overflow interrupt

4.3 Packet Counter Interface

Interface	Signals / Function
System side	Memory-mapped register interface
Event input	PACKET_RECEIVED, PACKET_PROCESSED
Error input	CRC_ERROR, FIFO_OVERFLOW
Control	ENABLE, RESET, CLEAR
Data	Packet and error counter values
Status	Counter enable/status
Optional	Threshold or event interrupt

4.4 CRC Checker Interface

Interface	Signals / Function
System side	Memory-mapped control/status register interface
Data input	DATA_IN, DATA_VALID
Control	RESET, START, ENABLE
CRC configuration	Polynomial / initial CRC configuration
Status	BUSY, VALID, PASS, ERROR
Data output	Calculated CRC result
Optional	CRC completion/error interrupt

4.5 UART Interface

Interface	Signals / Function
System side	Memory-mapped register interface
Serial output	TX
Serial input	RX
Control	ENABLE, BAUD configuration
Data	TX_DATA, RX_DATA
Status	TX_READY, RX_VALID, BUSY
Optional	TX/RX interrupt

4.6 Timer Interface

Interface	Signals / Function
System side	Memory-mapped control/status interface
Control	ENABLE, RESET, COMPARE
Data	Counter value
Status	RUNNING, MATCH
Optional	Timer/compare interrupt

4.7 GPIO Interface

Interface	Signals / Function
System side	Memory-mapped register interface
Data	GPIO_INPUT, GPIO_OUTPUT
Control	Direction configuration
Status	Input/output state
Optional	GPIO event interrupt

5. Bus Protocol Details

The SoC uses a memory-mapped system bus to provide communication between the VeeR EL2 processor and the memories, mandatory peripherals, and additional IP blocks. The bus interconnect decodes the processor address and selects the corresponding slave device.

5.1 Bus Architecture

- The VeeR EL2 processor acts as the bus master for instruction and data transactions.
- The interconnect decodes the address and generates the appropriate slave select signal.
- Instruction Memory, Data Memory, UART, Timer, GPIO, Packet FIFO, Packet Counter, and CRC Checker are connected as bus slaves.

- Read transactions return data from the selected slave to the processor.
- Write transactions transfer processor-generated data to the selected slave.
- The interconnect provides the required response and completion signals for each transaction.

5.2 Read Transaction

A typical read transaction follows these steps:

1. The processor provides the target address and read control information.
2. The bus decodes the address and selects the corresponding slave.
3. The selected slave provides the requested data.
4. The bus routes the read data back to the processor.
5. The transaction completes when the bus indicates a valid response.

5.3 Write Transaction

A typical write transaction follows these steps:

1. The processor provides the target address, write control information, and write data.
2. The bus decodes the address and selects the corresponding slave.
3. The selected slave accepts the write data.
4. The slave updates the addressed register or memory location.
5. The transaction completes with the appropriate bus response.

5.4 Peripheral Access

All additional IPs are accessed using memory-mapped registers. For example, the processor can read the Packet Counter registers to obtain packet statistics, access the Packet FIFO registers to control or monitor buffering, and read the CRC Checker registers to obtain the CRC result and status.

5.5 Bus Error Handling

An invalid or unmapped address should generate an appropriate bus error response rather than selecting any valid peripheral. This prevents unintended access to system resources.

5.6 Protocol Configuration

The exact bus protocol, signal widths, timing, and handshake mechanism will be finalized after confirming the bus configuration of the faculty-provided VeeR EL2 RTL. The reference architecture uses AHB-Lite with a shared bus, address decoding, read-data multiplexing, and HREADY-based transaction completion.

6. Architecture Summary

The proposed **RISC-V VeeR EL2 Based Hardware Packet Buffering and Data Integrity SoC** integrates the VeeR EL2 RISC-V processor with instruction memory, data memory, UART, Timer, GPIO, and three additional IP blocks: **Packet FIFO, Packet Counter, and CRC Checker**. The processor controls the system through a memory-mapped system bus, while the Packet FIFO buffers incoming data, the Packet Counter monitors packet activity, and the CRC Checker detects corrupted packets. Packet data is provided by an RTL testbench and follows the **Packet Source → FIFO → CRC Checker** processing path. The RISC-V application reads the hardware status and results and reports them through UART. The complete architecture will be verified through **RTL simulation and embedded C execution**, including normal packets, CRC errors, multiple packets, and FIFO overflow conditions.